# UNIVERSIDAD RAFAEL LANDÍVAR

Campus San Alberto Hurtado, S.J. de Quetzaltenango

Facultad de Ingeniería | Departamento de Ingeniería en Informática y Sistemas

**Curso:** Manejo e Implementación de Archivos | **Ciclo:** 2026

| | | | |
|---|---|---|---|
| **Laboratorio:** | Laboratorio No. 1 - Aplicación de Escritorio con Gestión de Configuración de Usuario | **Fecha de Entrega:** | Agosto - Octubre 2026 |
| **Estudiante:** | Josué Soch (Shonsoch) | **Tecnología:** | Python 3.14 / Tkinter & ttk / JSON / Git |

## 1. Selección y Justificación Técnica del Formato de Persistencia

Para la solución solicitada se adoptó el estándar **JSON (JavaScript Object Notation)**. Para fundamentar la elección con criterios técnicos de peso, se realizó una contrastación rigurosa frente a formatos comunes de almacenamiento de configuración:

| Criterio Técnico | JSON (Elegido) | CSV (Descartado) | XML (Descartado) | Texto Plano / INI (Descartado) |
|---|---|---|---|---|
| **Preservación de Tipos** | Excelente (Tipado nativo para números, cadenas, booleanos). | Deficiente (Todo es texto plano; exige casteo manual). | Regular (Requiere conversión manual o esquemas XSD pesados). | Limitada (Solo pares clave-valor planos). |
| **Parseo y Rendimiento** | Óptimo (Módulo estándar `json` implementado en C). | Rápido pero limitado a estructuras tabulares 2D sin jerarquías. | Pesado (Construcción de árboles DOM y alto consumo de RAM). | Rápido pero propenso a inconsistencias sintácticas. |
| **Legibilidad Humana** | Excelente (Estructura clara de diccionarios, limpia y directa). | Media (Complejo de leer con rutas con diagonales o comas). | Baja (Sobrecarga de etiquetas repetidas `<tag>`). | Buena pero limitada en jerarquías. |
| **Robustez ante Corrupción** | Alta (Falla estrictamente ante truncamiento con `JSONDecodeError`). | Muy baja (Puede leer filas cortadas silenciosamente). | Alta pero la recuperación ante errores de etiquetas es compleja. | Baja (Ignora líneas incompletas sin advertencia). |

**Conclusión técnica:** JSON permite representar la configuración de forma jerárquica con tipos de datos nativos (el tamaño de fuente se serializa como número entero `16` y no como cadena `"16"`). Además, su especificación estricta garantiza que ante un apagón o escritura incompleta, el parser detecta la falla inmediatamente, permitiendo que el sistema regenere los valores seguros predeterminados.

## 2. Implementación de Escritura Segura y Respaldo Atómico

Para garantizar que el archivo nunca quede corrupto o a medio escribir ante un cierre abrupto de la aplicación, se implementó el patrón de **escritura atómica en tres fases**:

1. **Copia de Respaldo Preventiva (`config.bak`):** Si ya existe una configuración previa válida en disco, se duplica usando `shutil.copy2()`, asegurando que si el proceso de guardado colapsa, la versión anterior permanece intacta.

2. **Escritura en Archivo Temporal (`config.tmp`):** Los nuevos parámetros se vuelcan primero a un archivo temporal intermedio con codificación forzada en UTF-8 (`ensure_ascii=False`).
3. **Reemplazo Atómico (`os.replace`):** Una vez cerrado y consolidado el temporal en disco, se ejecuta `os.replace(TEMP_FILE, CONFIG_FILE)`. En los sistemas de archivos modernos (NTFS, ext4), esta llamada es atómica a nivel de inodo/sistema de archivos: el puntero se sustituye en un solo paso, eliminando cualquier ventana temporal donde el archivo pueda quedar dañado.

### Fragmento del Código Implementado (`config_manager.py`)

```
def save_config(self, new_config):
    self.config.update(new_config)

    # 1. Respaldo (backup) de la configuración previa
    if os.path.exists(CONFIG_FILE):
        if os.path.exists(BACKUP_FILE):
            try:
                os.chmod(BACKUP_FILE, stat.S_IWRITE)
            except Exception:
                pass
        try:
            shutil.copy2(CONFIG_FILE, BACKUP_FILE)
        except PermissionError:
            print("ERROR: Sin permisos para crear el archivo de respaldo (.bak).")
            return False

    # 2. Escritura segura en archivo temporal intermedio
    try:
        with open(TEMP_FILE, 'w', encoding='utf-8') as f:
            json.dump(self.config, f, indent=4, ensure_ascii=False)
    except PermissionError:
        print("ERROR: Sin permisos para escribir en el archivo temporal (.tmp).")
        return False

    # 3. Reemplazo atómico seguro del archivo final
    try:
        os.replace(TEMP_FILE, CONFIG_FILE)
        print("INFO: Configuración guardada y actualizada correctamente.")
        return True
    except PermissionError:
        print("ERROR: Sin permisos para reemplazar el archivo de configuración original.")
        return False
```

## 3. Manejo de Errores y Degradación Controlada (Evidencia)

El sistema captura explícitamente y sin tracebacks los tres escenarios de contingencia exigidos por el laboratorio:

- **Caso 1: Archivo ausente:** Al no encontrar `config.json`, el gestor utiliza `DEFAULT_CONFIG` sin fallar, permitiendo que la interfaz inicie con parámetros de fábrica.
- **Caso 2: Archivo corrupto o inválido:** Si el JSON está truncado o mal formado, se atrapa `json.JSONDecodeError`, se notifica el error por consola y se degrada pacíficamente a la configuración base.
- **Caso 3: Falta de permisos (Lectura/Escritura):** Se captura `PermissionError` en cada operación de E/S. En lectura degrada a los valores predeterminados; en escritura retorna `False` y despliega un diálogo informativo en la interfaz gráfica.

### Evidencia de Ejecución de Pruebas Automatizadas (`logs.txt`)

```
============================================================
BATERÍA DE PRUEBAS DE PERSISTENCIA Y MANEJO DE ERRORES
============================================================

------------------------------------------------------------
--- CASO 1: ARCHIVO AUSENTE
------------------------------------------------------------
INFO: Archivo de configuración no encontrado. Usando valores por defecto.
Configuración activa actual (idioma): es/es-ES
Comportamiento: Degradación suave a valores por defecto sin excepción.

------------------------------------------------------------
--- CASO 2: ARCHIVO CORRUPTO / FORMATO INVÁLIDO
------------------------------------------------------------
ERROR: Archivo de configuración corrupto o con formato inválido. Degradando a valores por defecto.
Configuración activa actual (nombre_usuario): José Núñez
Comportamiento: Detección de JSONDecodeError y fallback a valores por defecto.

------------------------------------------------------------
--- CASO 3: SIN PERMISOS DE ESCRITURA
------------------------------------------------------------
INFO: Configuración cargada exitosamente.
Intentando guardar nueva configuración en archivo protegido...
ERROR: Sin permisos para reemplazar el archivo de configuración original.
Resultado del guardado controlado: Fallido (capturado controladamente)

------------------------------------------------------------
--- CASO 4: CODIFICACIÓN UTF-8 (Tildes y Carácter Ñ)
------------------------------------------------------------
INFO: Configuración cargada exitosamente.
INFO: Configuración guardada y actualizada correctamente.
INFO: Configuración cargada exitosamente.
Valor guardado y recuperado: José Núñez de la Peña
Validación de coincidencia exacta UTF-8: True

============================================================
TODAS LAS PRUEBAS FINALIZADAS EXITOSAMENTE SIN TRACEBACKS
============================================================
```

## 4. Manejo de Codificación de Caracteres UTF-8

Para garantizar la preservación fiel de tildes, diacríticos y la letra **ñ** en campos como `nombre_usuario` e `idioma`, tanto la apertura del archivo (`open(..., encoding='utf-8')`) como la serialización (`ensure_ascii=False`) fuerzan el estándar UTF-8 real en bytes.

**Archivo Generado: `config.json`**

```
{
    "nombre_usuario": "José Núñez de la Peña",
    "tema_interfaz": "oscuro",
    "idioma": "es/es-ES",
    "tamaño_fuente": 16,
    "color_barra_menu": "#2b5b84",
    "color_letra": "#ffffff",
    "foto_perfil": "C:/Usuarios/Imágenes/foto_diseño.png"
}
```

**Archivo de Respaldo: `config.bak`**

```
{
    "nombre_usuario": "José Núñez de la Peña",
    "tema_interfaz": "oscuro",
    "idioma": "es/es-ES",
    "tamaño_fuente": 14,
    "color_barra_menu": "#2b5b84",
    "color_letra": "#ffffff",
    "foto_perfil": "C:/Usuarios/Imágenes/foto_diseño.png"
}
```

## 5. Características de la Interfaz Gráfica (Tkinter / ttk)

**1. Menú Principal Jerárquico:**

- **Archivo:** Submenús simulados (*Nuevo*, *Abrir*, *Salir*) con retroalimentación no intrusiva en pantalla.
- **Edición:** Opciones simuladas (*Deshacer*, *Copiar*).
- **Ver:** Control de ventana (*Minimizar*, *Maximizar*, *Restaurar*, *Pantalla Completa*).
- **Configuración:** Abre la ventana modal de ajustes.

**2. Ventana de Configuración Avanzada:**

- Selectores gráficos de color (Barra de Menú y Tipografía) mediante `colorchooser`.
- Selector de imagen de Avatar mediante `filedialog` y renderizado directo en la ventana central.
- Internacionalización dinámica completa (Español / Inglés) que traduce toda la interfaz en tiempo real.
- Botón **Aplicar** para guardar cambios sin cerrar la ventana de ajustes, y botón **Restaurar Fábrica**.